Pandas Tutorial 5: GroupBy and Aggregation

In this tutorial, students will learn how to group data and perform aggregation operations. GroupBy is one of the most powerful features of Pandas and is heavily used in Exploratory Data Analysis.

---

1. What is GroupBy

GroupBy is used to group rows based on one or more columns and then perform calculations on each group.

In simple words, it means:

Split the data into groups. Apply a function. Combine the results.

This concept is very important in real world data analysis.

---

2. Basic GroupBy Syntax

```
df.groupby("ColumnName")
```

Example:

```
df.groupby("Department")
```

This groups the dataset based on the Department column.

However, grouping alone does not show results. We must apply an aggregation function.

---

3. Using Aggregation Functions

Common aggregation functions:

- sum()
- mean()
- count()
- min()
- max()
- median()

Example:

```
df.groupby("Department")["Salary"].mean()
```

This calculates the average salary for each department.

---

4. **GroupBy with Multiple Columns**

We can group by more than one column.

```
df.groupby(["Department", "Gender"])["Salary"].mean()
```

This groups data first by Department and then by Gender.

---

5. **Using Multiple Aggregation Functions**

We can apply multiple functions at the same time.

```
df.groupby("Department")["Salary"].agg(["mean", "max", "min"])
```

This returns mean, maximum, and minimum salary for each department.

---

6. **Using agg with Dictionary**

We can apply different functions to different columns.

```
df.groupby("Department").agg({
    "Salary": "mean",
    "Age": "max"
})
```

This calculates:

- Mean salary per department.
- Maximum age per department.

---

7. **Resetting Index After GroupBy**

GroupBy results usually set grouped column as index.

To convert it back to normal column:

```
df.groupby("Department")["Salary"].mean().reset_index()
```

This makes the output cleaner and easier to use.

---

8. **Sorting Grouped Results**

After grouping, we can sort results.

```
df.groupby("Department")["Salary"].mean().sort_values(ascending=False)
```

This sorts departments by average salary in descending order.

---

9. **Counting Values Using GroupBy**

To count number of records in each group:

```
df.groupby("Department").count()
```

Or count specific column:

```
df.groupby("Department")["Name"].count()
```

This is useful for understanding distribution of categories.

---

10. **Practical Example**

Example dataset structure:

Name | Department | Salary | Age

Using GroupBy:

```
df.groupby("Department")["Salary"].mean()
```

This tells which department has higher average salary.

---

11. **Conclusion of Tutorial 4**

In this tutorial, students have learned:

1. What GroupBy means.
2. How to group data by one or more columns.
3. How to apply aggregation functions.
4. How to use multiple aggregation functions.
5. How to reset index after grouping.
6. How to sort grouped results.

GroupBy and aggregation are core skills for performing meaningful Exploratory Data Analysis.

In the next tutorial, we will learn about merging, joining, and concatenating DataFrames.

This completes Pandas Tutorial 4.